

Advanced Algorithms and Complexity

Huffman Coding: Proof of Optimality

Lecture Scribe

Scribe: Vardan Sharma and Prajwal Cheela

Professor: Aniket Basu Roy

1. Problem Setup

Huffman coding is used to construct an efficient **prefix code**. The objective is to assign shorter codewords to symbols that occur frequently and longer codewords to symbols that occur less frequently.

The main goal of this lecture is to prove that the code produced by Huffman's algorithm is not just a good code, but an **optimal prefix code**.

Let

$$S = \{a_1, a_2, \dots, a_n\}$$

be the alphabet. For every symbol $x \in S$, let

$$f(x) \in (0, 1)$$

denote its frequency in the input. Therefore,

$$\sum_{x \in S} f(x) = 1.$$

A binary code assigns a sequence of 0s and 1s to every symbol. We write the code as

$$\gamma : S \rightarrow \{0, 1\}^*.$$

For a symbol x , the length of its codeword is denoted by

$$|\gamma(x)|.$$

1.1 Prefix Codes

A code is called a **prefix code** if no codeword is a prefix of another codeword.

This property is important because it makes the encoded sequence unambiguous. Once a complete codeword is encountered, it is possible to determine the corresponding symbol without requiring separators.

1.2 Average Bits per Letter

The quality of a prefix code is measured using the **average number of bits per letter**, abbreviated as ABL.

It is defined by

$$\text{ABL}(\gamma) = \sum_{x \in S} f(x) |\gamma(x)|.$$

The term

$$f(x) |\gamma(x)|$$

represents the contribution of symbol x to the average number of bits.

Therefore, the optimization problem is

$$\boxed{\text{Find a prefix code that minimizes } \text{ABL}(\gamma).}$$

An encoding achieving the minimum possible ABL is called an **optimal prefix code**.

The basic intuition is simple: if a symbol occurs very frequently, giving it a short code has a large effect on the average. Therefore, frequent symbols should tend to have shorter codewords.

2. Prefix Codes as Binary Trees

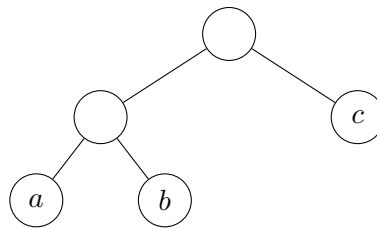
It is useful to represent a prefix code using a binary tree.

We use the convention:

- a left edge represents bit 0;
- a right edge represents bit 1.

Each symbol is stored at a leaf. The codeword of a symbol is the sequence of edge labels on the path from the root to its leaf.

For example:



The length of a codeword is therefore exactly the depth of the corresponding leaf:

$$\boxed{|\gamma(x)| = \text{depth}_T(x).}$$

Hence, we can rewrite the ABL as

$$\boxed{\text{ABL}(T) = \sum_{x \in S} f(x) \text{depth}_T(x).}$$

Thus, instead of directly searching over binary strings, we can think of the problem as finding a binary tree that minimizes the frequency-weighted sum of the depths of its leaves.

Goal: Find a binary tree T that minimizes

$$\text{ABL}(T) = \sum_{x \in S} f(x) \text{depth}_T(x).$$

This tree representation is the basis of the optimality proof.

3. Properties of an Optimal Tree

We now establish structural properties that an optimal tree must satisfy.

These properties will eventually justify the greedy choice made by Huffman's algorithm.

3.1 An Optimal Tree is Full

Definition 1. A binary tree is called **full** if every internal node has exactly two children.

Lemma 1. *The binary tree corresponding to an optimal prefix code is full.*

Proof. Suppose an optimal tree contains an internal node u having only one child v .

We can remove u and connect v directly to the parent of u . If u is the root, then v becomes the new root.

After this operation, every leaf in the subtree rooted at v moves one level closer to the root. Therefore, the depth of each of these leaves decreases by one.

Since every frequency is positive, decreasing the depth of at least one leaf strictly decreases

$$\text{ABL}(T) = \sum_{x \in S} f(x) \text{depth}_T(x).$$

This contradicts the assumption that the original tree was optimal.

Therefore, an optimal tree cannot contain an internal node with only one child.

Hence,

Every internal node of an optimal tree has two children.

Thus, an optimal tree is full.

□

3.2 Frequency and Depth

The next lemma tells us how the frequency of a symbol is related to its depth in an optimal tree.

Lemma 2. *Let u and v be two leaves of an optimal tree such that*

$$\text{depth}(u) < \text{depth}(v).$$

If u contains symbol y and v contains symbol z , then

$$f(y) \geq f(z).$$

Proof. Suppose instead that

$$f(y) < f(z).$$

Thus, the less frequent symbol y is at the shallower leaf, while the more frequent symbol z is at the deeper leaf.

Now exchange the labels y and z .

The change in ABL is

$$(\text{depth}(v) - \text{depth}(u))(f(y) - f(z)).$$

Since

$$\text{depth}(u) < \text{depth}(v),$$

we have

$$\text{depth}(v) - \text{depth}(u) > 0.$$

Also,

$$f(y) < f(z),$$

so

$$f(y) - f(z) < 0.$$

Therefore,

$$(\text{depth}(v) - \text{depth}(u))(f(y) - f(z)) < 0.$$

Thus the exchange strictly decreases the ABL.

But this contradicts the optimality of the original tree.

Hence,

$$f(y) \geq f(z).$$

In words:

In an optimal tree, a higher-frequency symbol should not be deeper than a lower-frequency symbol.

□

4. The Key Sibling Claim

Let y^* and z^* denote the two symbols having the smallest frequencies in S .

The central structural fact used by Huffman's algorithm is:

Claim 1. *There exists an optimal tree in which y^* and z^* are sibling leaves.*

This claim is crucial because it tells us that merging the two least-frequent symbols is a safe greedy choice.

4.1 Proof of the Claim

Take an optimal tree and choose a leaf v having maximum depth.

Because the tree is full, the parent of v has exactly two children. One of them is v ; call the other child w .

We first show that w must also be a leaf.

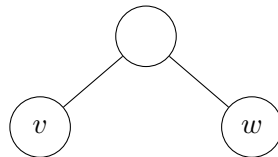
Suppose w were an internal node. Then w would have a child, and there would eventually be a leaf below w .

That leaf would be strictly deeper than v .

But v was chosen to be a deepest leaf. This is a contradiction.

Therefore, w is also a leaf.

Hence, v and w are sibling leaves at maximum depth:



Now consider the frequencies of the symbols occupying these deepest leaves.

Because v and w have maximum depth, any symbol whose current position is shallower can be exchanged with one of these leaves. By the frequency-depth exchange argument, moving a smaller-frequency symbol to a deeper position cannot increase the ABL.

In particular, we may choose an optimal labelling in which y^* occupies one of the deepest leaves.

The remaining minimum-frequency symbol z^* can similarly be placed at the other deepest leaf. Since v and w have the same depth, exchanging their labels does not change the ABL.

Therefore, there exists an optimal tree in which

y^* and z^* are sibling leaves.

Why this claim matters

Huffman's algorithm chooses the two least-frequent symbols and merges them. This is safe because we have just proved that there exists an optimal tree in which exactly these two symbols are siblings. Therefore, the greedy choice does not rule out an optimal solution.

5. Reducing the Problem

Since y^* and z^* can be siblings in an optimal tree, we can replace them by a single new symbol. Let this new symbol be the meta-symbol

$$\omega.$$

Its frequency is defined as

$$f(\omega) = f(y^*) + f(z^*).$$

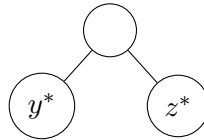
The reduced alphabet is

$$S' = (S - \{y^*, z^*\}) \cup \{\omega\}.$$

Since two symbols are removed and one symbol is added,

$$|S'| = |S| - 1.$$

Conceptually, we replace the two sibling leaves



by the single symbol

$$\omega.$$

The symbol ω represents the common parent of y^* and z^* .

The important point is that the reduced problem has exactly the same form as the original problem, but with one fewer symbol. Therefore, it can be solved recursively.

6. Huffman's Recursive Construction

The recursive construction used by Huffman's algorithm is:

1. If

$$|S| = 2,$$

- assign the codes 0 and 1 to the two symbols.
2. Otherwise, let y^* and z^* be the two lowest-frequency symbols.
 3. Create the meta-symbol ω with

$$f(\omega) = f(y^*) + f(z^*).$$

4. Construct the Huffman tree T' recursively for the reduced alphabet

$$S' = (S - \{y^*, z^*\}) \cup \{\omega\}.$$

5. Locate ω in T' and replace it by an internal node whose children are y^* and z^* .

The algorithm therefore repeatedly performs

$$\boxed{y^*, z^* \longrightarrow \omega}$$

until only two symbols remain.

The remaining question is why this recursive greedy construction always gives an optimal tree.

7. Relating the ABL of the Two Trees

Let T' be the tree obtained for the reduced alphabet S' .

Let T be the tree obtained by expanding the leaf ω in T' into the two leaves y^* and z^* .

All symbols other than y^* and z^* retain the same depth.

The two new leaves are one level deeper than ω . Therefore,

$$\text{depth}_T(y^*) = \text{depth}_{T'}(\omega) + 1$$

and

$$\text{depth}_T(z^*) = \text{depth}_{T'}(\omega) + 1.$$

Starting from

$$\text{ABL}(T) = \sum_{x \in S} f(x) \text{depth}_T(x),$$

separate the two terms corresponding to y^* and z^* :

$$\begin{aligned} \text{ABL}(T) &= f(y^*) \text{depth}_T(y^*) + f(z^*) \text{depth}_T(z^*) \\ &+ \sum_{x \in S - \{y^*, z^*\}} f(x) \text{depth}_T(x). \end{aligned}$$

Substituting the depths after expansion,

$$\begin{aligned} \text{ABL}(T) &= f(y^*)(\text{depth}_{T'}(\omega) + 1) \\ &\quad + f(z^*)(\text{depth}_{T'}(\omega) + 1) \\ &\quad + \sum_{x \in S - \{y^*, z^*\}} f(x) \text{depth}_{T'}(x). \end{aligned}$$

Collecting the first two terms,

$$\begin{aligned} \text{ABL}(T) &= (f(y^*) + f(z^*)) \text{depth}_{T'}(\omega) \\ &\quad + (f(y^*) + f(z^*)) \\ &\quad + \sum_{x \in S - \{y^*, z^*\}} f(x) \text{depth}_{T'}(x). \end{aligned}$$

Since

$$f(\omega) = f(y^*) + f(z^*),$$

we obtain

$$\begin{aligned} \text{ABL}(T) &= f(\omega) \text{depth}_{T'}(\omega) + f(\omega) \\ &\quad + \sum_{x \in S - \{y^*, z^*\}} f(x) \text{depth}_{T'}(x). \end{aligned}$$

The first term together with the summation is exactly $\text{ABL}(T')$.

Therefore,

$$\boxed{\text{ABL}(T) = \text{ABL}(T') + f(\omega).}$$

Equivalently,

$$\boxed{\text{ABL}(T') = \text{ABL}(T) - f(\omega).}$$

Interpretation:

When ω is expanded into y^* and z^* , each of these two symbols becomes one level deeper. Their combined frequency is $f(\omega)$, so the average number of bits increases by exactly $f(\omega)$.

This relationship is the algebraic link between the original problem and the reduced problem.

8. Proof of Optimality

We now prove the main theorem.

Theorem 1. *Huffman's algorithm produces a prefix code with minimum possible average number of*

bits per letter.

Proof. We use induction on the number of symbols in the alphabet.

Base Case

Suppose

$$|S| = 2.$$

Huffman's algorithm assigns one symbol the code

$$0$$

and the other symbol the code

$$1.$$

Thus both symbols have code length 1.

Since the two symbols are distinct, a binary prefix code cannot give either symbol a code of length 0.

Therefore, one bit per symbol is the minimum possible, and the Huffman tree is optimal.

Induction Hypothesis

Assume that Huffman's algorithm produces an optimal prefix code for every alphabet containing $k - 1$ symbols.

Induction Step

Now consider an alphabet S containing

$$|S| = k$$

symbols.

Let y^* and z^* be the two lowest-frequency symbols.

Huffman merges them into the meta-symbol ω :

$$f(\omega) = f(y^*) + f(z^*).$$

The resulting alphabet is

$$S' = (S - \{y^*, z^*\}) \cup \{\omega\},$$

and therefore

$$|S'| = k - 1.$$

By the induction hypothesis, Huffman's algorithm produces an optimal tree T' for the reduced alphabet S' .

Expanding ω into y^* and z^* produces a tree T for the original alphabet.

We must show that T is optimal.

Contradiction

Suppose, for contradiction, that T is not optimal.

Then there exists an optimal tree Z such that

$$\boxed{\text{ABL}(Z) < \text{ABL}(T).}$$

From the key sibling claim, we can choose an optimal tree Z in which y^* and z^* are sibling leaves.

Now collapse these two sibling leaves into their common parent and label that parent by ω .

The resulting tree is a valid tree Z' for the reduced alphabet S' .

For Z and Z' , the same ABL relationship gives

$$\boxed{\text{ABL}(Z') = \text{ABL}(Z) - f(\omega).}$$

For the Huffman trees T and T' we have

$$\boxed{\text{ABL}(T') = \text{ABL}(T) - f(\omega).}$$

We assumed that

$$\text{ABL}(Z) < \text{ABL}(T).$$

Subtracting the same quantity $f(\omega)$ from both sides gives

$$\text{ABL}(Z) - f(\omega) < \text{ABL}(T) - f(\omega).$$

Hence,

$$\boxed{\text{ABL}(Z') < \text{ABL}(T').}$$

But T' is an optimal Huffman tree for the reduced alphabet S' , which contains $k - 1$ symbols.

By the induction hypothesis, no tree for S' can have an ABL smaller than $\text{ABL}(T')$.

Thus,

$$\text{ABL}(Z') < \text{ABL}(T')$$

is a contradiction.

Therefore, our assumption that T is not optimal must be false.

Hence,

Huffman's algorithm produces an optimal prefix code.

□

9. Time Complexity

Let

$$k = |S|$$

be the number of symbols.

Huffman's algorithm performs

$$k - 1$$

merging steps.

9.1 Straightforward Implementation

If we scan the current set of symbols at every step to find the two minimum-frequency symbols, one iteration can take $O(k)$ time.

Since there are $O(k)$ iterations, the total running time is

$$O(k^2).$$

9.2 Using a Min-Heap

We can improve the implementation using a priority queue, or specifically a **min-heap**, with frequency as the key.

At every iteration we:

1. extract the minimum-frequency symbol;
2. extract the next minimum-frequency symbol;
3. merge them;
4. insert the new meta-symbol.

Each heap operation takes

$$O(\log k)$$

time.

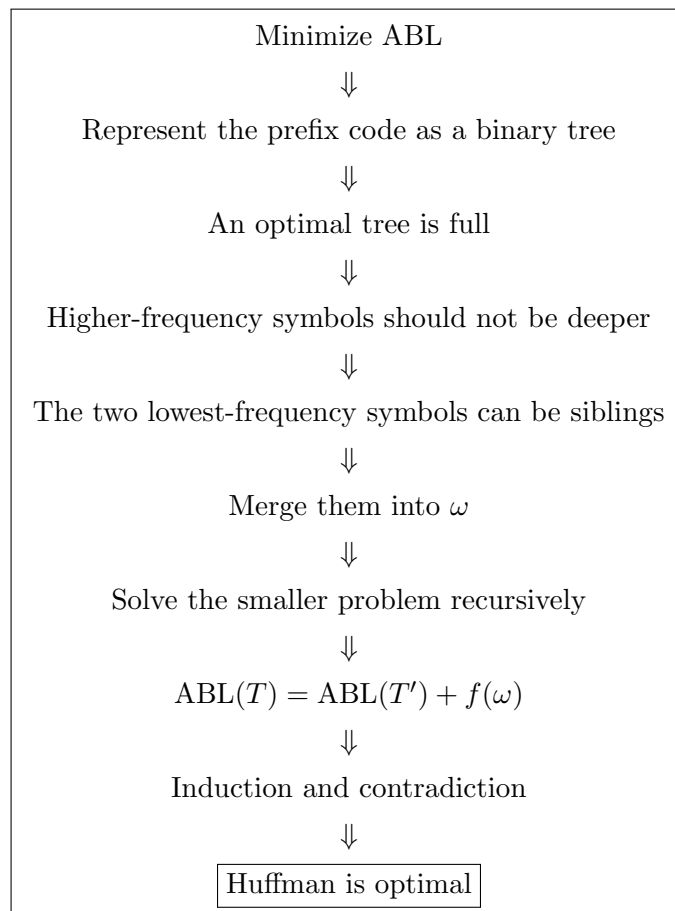
There are $O(k)$ iterations, so the total running time becomes

$$O(k \log k).$$

Thus, the min-heap implementation gives the standard efficient running time for Huffman's algorithm.

10. Conclusion

The proof of Huffman's optimality can be summarized by the following chain of ideas:



The key idea is the sibling claim:

There exists an optimal tree in which the two least-frequent symbols are siblings.

This is what makes Huffman's greedy choice safe. Once these two symbols are merged, the remaining problem is a smaller instance of the same problem, allowing induction to complete the proof.

Finally, using a min-heap, Huffman's algorithm can be implemented in

$$O(k \log k) \text{ time.}$$